

Raspberry Pi Pico L293D Wokwi User Manual

Little Daisies

November 27, 2025

1 Introduction

This project demonstrates the control of DC motors (visualized via LEDs) using a Raspberry Pi Pico and a simulated L293D motor driver in the Wokwi environment. The system utilizes two push buttons to toggle the "motor" direction between Forward and Backward.

**You can reach the wokwi assignment with this link:

<https://wokwi.com/projects/448766450325210113>

2 System Pinout and Wiring Diagrams

This section provides a visual representation of the connections between the Raspberry Pi Pico, the L293D Motor Driver, and the peripheral components, matching the source code logic.

2.1 LEDs (H-Bridge Direction Pair)

LED1 (Channel 1)

Component	Connect to
LED1 + (anode)	L293D OUT1
LED1 - (cathode)	220 Ω resistor $\rightarrow$ GND

LED2 (Channel 2)

Component	Connect to
LED2 + (anode)	L293D OUT2
LED2 - (cathode)	220 Ω resistor $\rightarrow$ GND

2.2 L293D Inputs

Pico W Pin	L293D Pin
GP12	IN3
GP11	IN4
GP13	EN1/EN2 (enable pin, PWM)

2.3 Buttons

Button	Wiring
FORWARD button	GP14 → button → GND (internal pull-up)
BACKWARD button	GP15 → button → GND

2.4 L293D Power Connections

Connection	Notes
VCC1 (pin 16)	Pico 5V
VCC2 (pin 8)	Pico 5V (LEDs have low current, safe)
GND pins	Connect all to Pico GND

2.5 Raspberry Pi Pico Connections

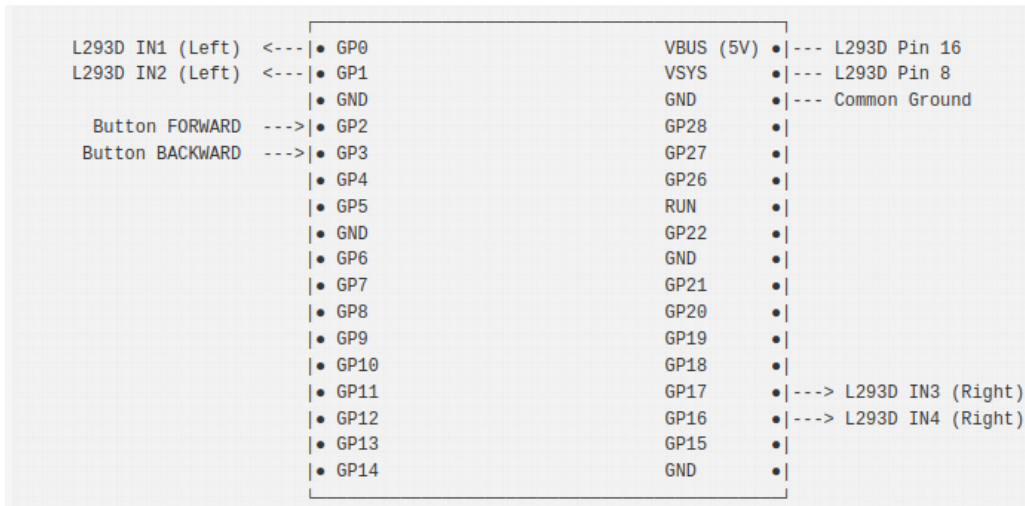


Figure 1: Pico W pinout diagram

2.6 L293D Motor Driver Connections

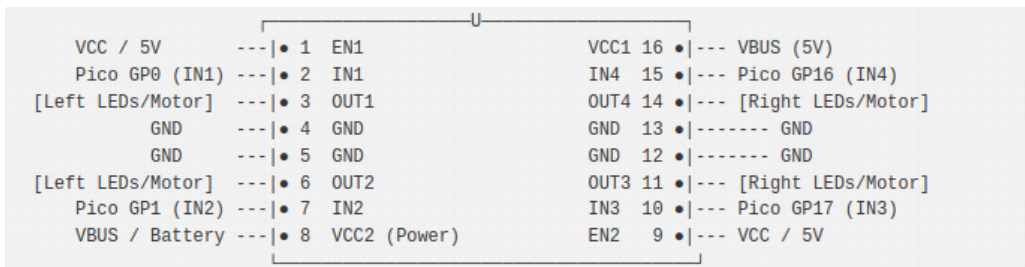


Figure 2: L293D Motor Driver pinout diagram

2.7 Wiring Notes

1. **Enable Pins (EN1, EN2):** In this configuration, Pins 1 and 9 of the L293D should be connected to 5V (High) to keep the motors enabled.


```

19     in2.low()
20     in3.low()
21     in4.low()
22
23 def move_forward():
24     # Polarity: Positive
25     in1.high()
26     in2.low()
27     in3.high()
28     in4.low()
29
30 def move_backward():
31     # Polarity: Negative (Reversed)
32     in1.low()
33     in2.high()
34     in3.low()
35     in4.high()
36
37 # --- Main Loop ---
38 print("System Ready. Press buttons to control direction.")
39
40 while True:
41     # Check if Button 1 (Forward) is pressed (Logic 0)
42     if btn_forward.value() == 0:
43         move_forward()
44         print("Moving Forward (Red LEDs)")
45
46     # Check if Button 2 (Backward) is pressed (Logic 0)
47     elif btn_backward.value() == 0:
48         move_backward()
49         print("Moving Backward (Blue LEDs)")
50
51     # If no button is pressed
52     else:
53         stop_all()
54
55     time.sleep(0.1) # Short delay to prevent CPU overload

```

Listing 1: MicroPython Motor Control Code

3.2 Wokwi Custom Chip Implementation (l293d.chip.c)

To simulate the L293D driver in Wokwi, a custom C-chip was implemented. This code handles the logic mapping between the Enable (EN), Input (IN), and Output (OUT) pins.

```

1 // Wokwi Custom Chip - L293D
2 // corrected l293d.chip.c
3
4 #include "wokwi-api.h"
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 typedef struct {
9     pin_t pin_en1, pin_in1, pin_out1, pin_in2, pin_out2;
10    pin_t pin_en2, pin_in3, pin_out3, pin_in4, pin_out4;
11 } chip_state_t;
12
13 void update_motor(pin_t en, pin_t in1, pin_t out1, pin_t in2, pin_t out2) {
14     // Read the Enable pin. If High, pass signals through.
15     if (pin_read(en)) {

```

```

16     pin_write(out1, pin_read(in1));
17     pin_write(out2, pin_read(in2));
18 } else {
19     // If disabled, turn outputs off (High Impedance)
20     pin_mode(out1, INPUT);
21     pin_mode(out2, INPUT);
22 }
23 }
24
25 void chip_pin_change(void *user_data, pin_t pin, uint32_t value) {
26     chip_state_t *chip = (chip_state_t*)user_data;
27     update_motor(chip->pin_en1, chip->pin_in1, chip->pin_out1, chip->pin_in2,
28                 chip->pin_out2);
29     update_motor(chip->pin_en2, chip->pin_in3, chip->pin_out3, chip->pin_in4,
30                 chip->pin_out4);
31 }
32
33 void chip_init() {
34     chip_state_t *chip = malloc(sizeof(chip_state_t));
35
36     // --- Assign Logical Pins to Variables ---
37     chip->pin_en1 = pin_init("EN1", INPUT);
38     chip->pin_in1 = pin_init("IN1", INPUT);
39     chip->pin_out1 = pin_init("OUT1", OUTPUT);
40     chip->pin_in2 = pin_init("IN2", INPUT);
41     chip->pin_out2 = pin_init("OUT2", OUTPUT);
42
43     chip->pin_en2 = pin_init("EN2", INPUT);
44     chip->pin_in3 = pin_init("IN3", INPUT);
45     chip->pin_out3 = pin_init("OUT3", OUTPUT);
46     chip->pin_in4 = pin_init("IN4", INPUT);
47     chip->pin_out4 = pin_init("OUT4", OUTPUT);
48
49     // Initialize Power/GND pins (passive in sim)
50     pin_init("VCC1", INPUT);
51     pin_init("VCC2", INPUT);
52     pin_init("GND", INPUT);
53
54     const pin_watch_config_t watch_config = {
55         .edge = BOTH,
56         .pin_change = chip_pin_change,
57         .user_data = chip,
58     };
59
60     // Watch for changes
61     pin_watch(chip->pin_en1, &watch_config);
62     pin_watch(chip->pin_in1, &watch_config);
63     pin_watch(chip->pin_in2, &watch_config);
64
65     pin_watch(chip->pin_en2, &watch_config);
66     pin_watch(chip->pin_in3, &watch_config);
67     pin_watch(chip->pin_in4, &watch_config);
68 }

```

Listing 2: Custom L293D Chip Simulation Code